NP-Completeness

Summary of the Course (So far)

Part-I: Basics [Time Complexity, Asymptotic analysis]

Sorting Algorithms

- Insertion Sort
- Merge sort
- Quick sort
- Heap Sort

Part-II:

- Divide and Conquer
- Dynamic Programming
- Greedy Algorithms

Part-III:

Graph Algorithms

- BFS & its applications
- DFS & its applications
- MST : Prim's & Kruskal's
- Shortest Paths
  - Dijkstra's Algorithm
  - Bellmanford's Algorithm
  - Floyd Warshall's Algorithm

Part - IV

- NP-completeness



today

Part - V :

- Approximation Algorithms
- Randomized Algorithms

Story so far

For a wide range of Problems we have developed **Polynomial time** algorithms.

On input of size n, their worst-case running time is $O(n^c)$ for some constant c

Q: Whether all Problems can be solved in Polynomial time?

Ans. No,

More on this in TOC course

For example "Halting Problem" can not be Solved by any Computer, no matter how much time we allow.

There are also problems that can be solved, but not in time $O(n^k)$ for any constant k.

"Generally, we think of Problems that are solvable in Polynomial time algorithms as being tractable, or easy and Problems that require Superpolynomial time as being intractable or hard."

In this class / topic

We are not worried about exact running times.

We only interested whether the running time is Polynomial or Super-Polynomial.

Decision Problem

A decision Problem is a Computational Problem that can be Posed as a YES-NO question of the input values.



Examples

① Input: a natural number n

Q: Is n prime?

② Input: a natural numbers x and y

Q: Does x divide y?

Shortest Path Problem

Input: An undirected graph G and two vertices s and t of G.

Output: Find the shortest s to t path.

Shortest Path Problem (Optimization version)

Input: An undirected graph G and two vertices s and t of G.

Question: Find the shortest s to t path.

Shortest Path Problem: (Decision version)

Input: An undirected graph G and two vertices s and t of G, and an integer k?

Q: Is there a shortest s to t path of length at most k?

Bipartite Graph

Input: Graph G

Q: : Is G bipartite?

MST

Input: Weighted graph G, integer k

Q: Is there a spanning tree of total weight $\leq k$?

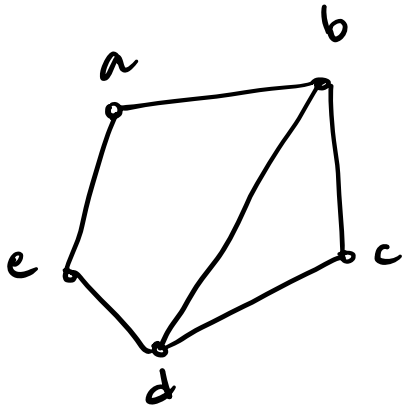
Next we look at few graph theoretic Problems.

- VERTEX COVER
- INDEPENDENT SET
- CLIQUE

Graph theoretic Problems

Vertex cover :

A Vertex cover of a graph G is a set $X \subseteq V(G)$ that contains at least one endpoint of every edge.



The size of a Vertex cover is the number of vertices it contains.

The Vertex cover problem is to find a Vertex Cover of minimum size in a graph G.

The size of a minimum size Vertex Cover of G is denoted by $\beta(G)$ or $MVC(G)$

Warmup examples:

Find $\beta(G)$ for the following graphs

- Complete graphs
- Path (P_n)
- Cycle (C_n)
- TREES
- BIPARTITE GRAPHS

VERTEX COVER PROBLEM (Optimization Version)

Input: A graph G (undirected)

Q: Find a vertex cover of minimum size in G.

(or)

Find $\beta(G)$

Brute-force Algorithm for V.C:

VC(G)

```
1.  MVC = |V(G)|
2.  For each X ⊆ V(G)
3.      For all e = uv ∈ E(G)
4.          if both u and v ∉ E(G)
5.              return False
6.      if |X| < MVC
7.          MVC = |X|
8.  return MVC
```

Running time : $O(2^n m)$

VERTEX COVER (Decision version)

Input: A graph G and an integer k

Question: Does G contain a vertex cover of size at most k?

VERTEX COVER

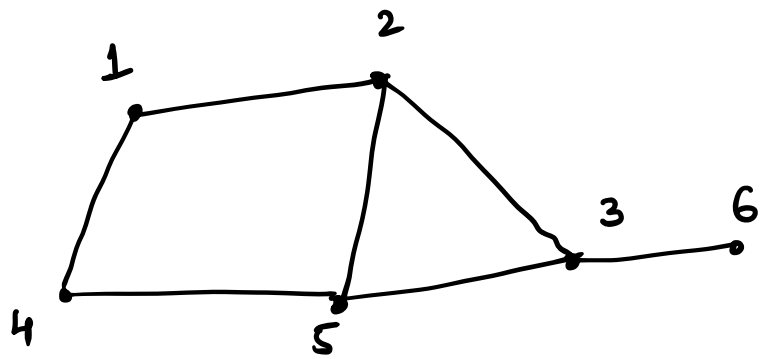
Decision Version Vs Optimization Version

Q: Which one is easier to solve?

Independent set :

In a graph $G=(V,E)$, we say subset $I \subseteq V$ is independent if no two vertices in I are adjacent.

The **independent set Problem** is to find the largest independent set in a graph G.



The largest size independent set $= \{2, 4, 6\}$

Size $= 3$.

The size of a maximum independent set in G is denoted by $\alpha(G)$.

Independent set (Decision version)

Input: A graph G and an integer k

Question: Does G contain an independent set of size at least k?

Warm up examples:

Find $\alpha(G)$ for the following graphs

- Complete graphs
- Path (P_n)
- Cycle (C_n)

INDEPENDENT SET Problem

Decision version vs Optimization version

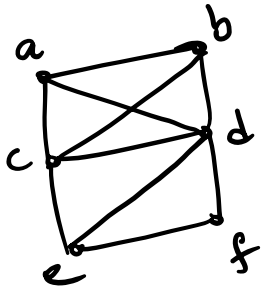
Q: Which one is easier to solve?

CLIQUE in a graph:

A subset $C \subseteq V(G)$ is called a clique in a graph G if for all $u, v \in C$, $uv \in E(G)$ (i.e., C is a complete subgraph of G).

The **CLIQUE Problem** is to find the size of a largest clique in a given graph (optimization version).

Ex:-



$C = \{a, b, c, d\}$ is a clique of size 4 (also largest)

The size of a largest clique in a graph G is denoted by $\omega(G)$. (also called clique number)

CLIQUE Problem [Decision version]

Input: Graph G, and an integer k

Question: Decide whether G has a clique of size k?

Warmup examples:

Find $\omega(G)$ for the following graphs

- Complete graphs
- Path (P_n)
- Cycle (C_n)

CLIQUE

Decision Version vs Optimization Version

Q: Which one is easier to solve?

Given a graph G,

Q: ① Is there a relation between Vertex Cover and Independent Sets in G?

True | False:

① If X is a vertex cover of G, then

$I = V(G) - X$ is an independent set of G

② If I is an independent set of G then

$X = V(G) - I$ is a vertex cover of G.

Theorem:

X is a minimum size Vertex Cover of G

if and only if

$I = V(G) - X$ is a maximum size independent set in G.

In otherwords.

Size of MVC + Size of Maximum ind set

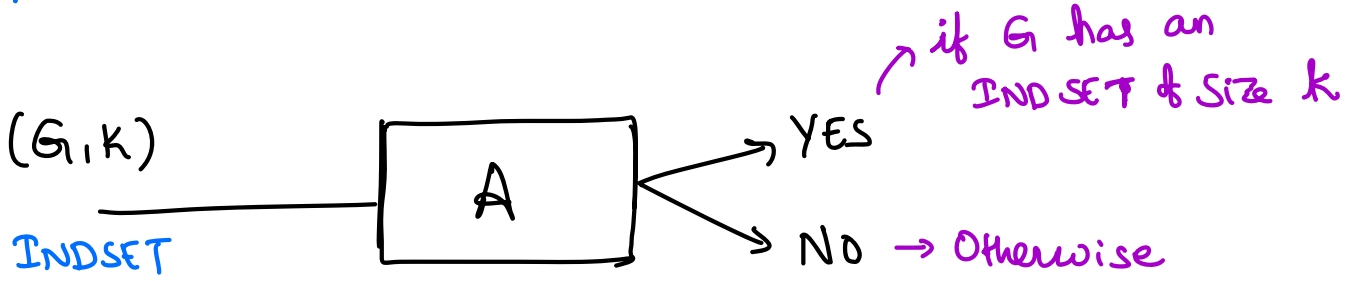
= Number of vertices.

or

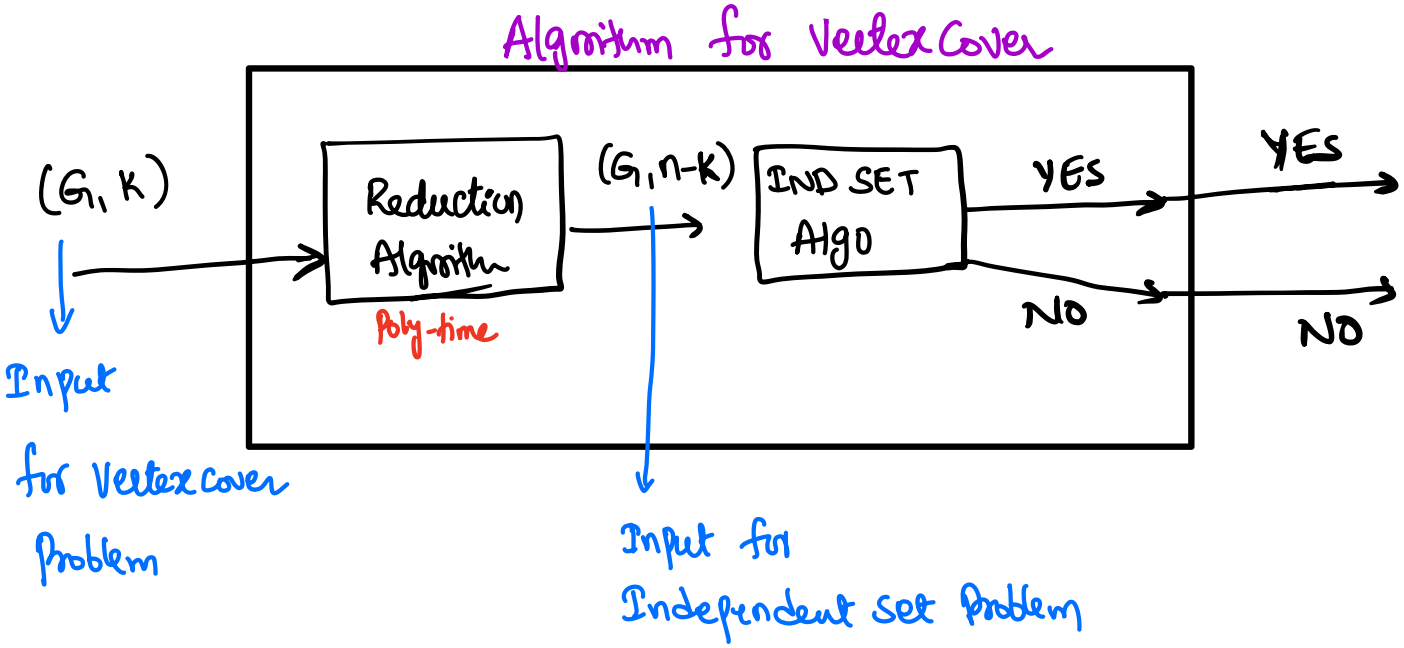
$$\alpha(G) + \beta(G) = n$$

Where n = # of vertices in G

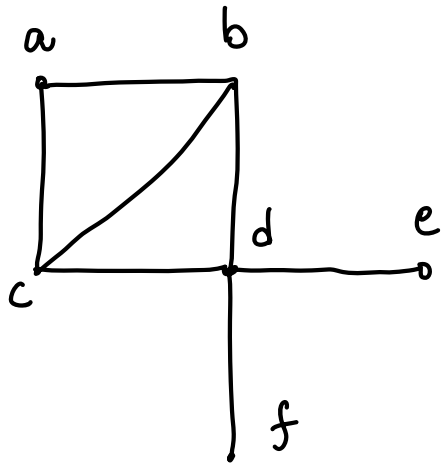
Suppose you have an algorithm A that solves Independent set Problem



Q: Can we use A to solve VERTEX COVER Problem.


Algorithm for Vertex Cover
(G, K)
Reduction Algorithm
Poly-time
(G, n-K)
IND SET Algo
YES
YES
NO
NO
Input for Vertex cover Problem
Input for Independent set Problem

Warmup - Questions



Graph G

Q: Find $\beta(G)$ and $\alpha(G)$

Given a graph G,

Clique

Q: ① Is there a relation between ~~Vertex Cover~~

and Independent Set in G?

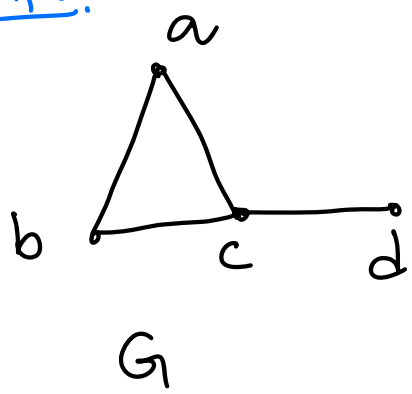
Complement of a graph:

The Complement of G is denoted by $\overline{G}$, defined as follows.

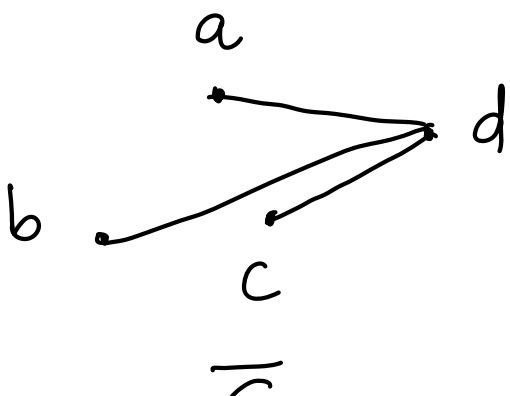
$$V(\overline{G}) = V(G)$$

$$E(\overline{G}) = \{ uv \mid u, v \in V(G) \,\&\, uv \notin E(G) \}$$

Example:



G



$\overline{G}$

True | False :

If I is an independent set in G

then I is a clique in $\overline{G}$

Theorem:

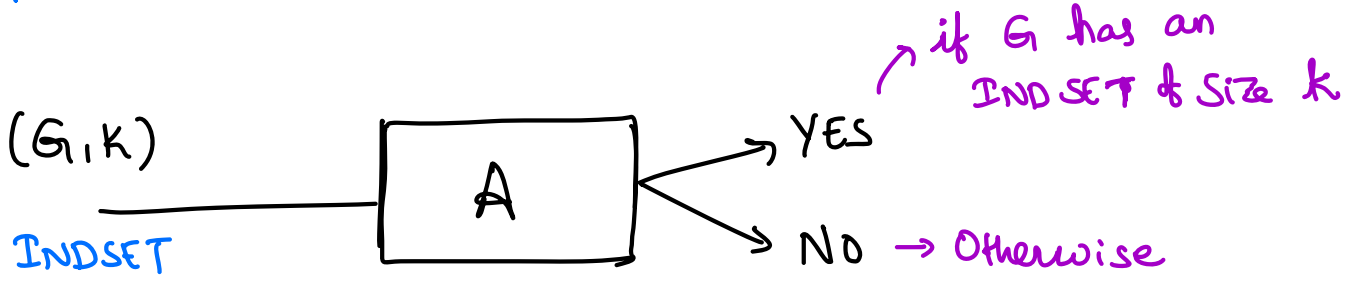
I is a maximum size independent set in G

if and only if

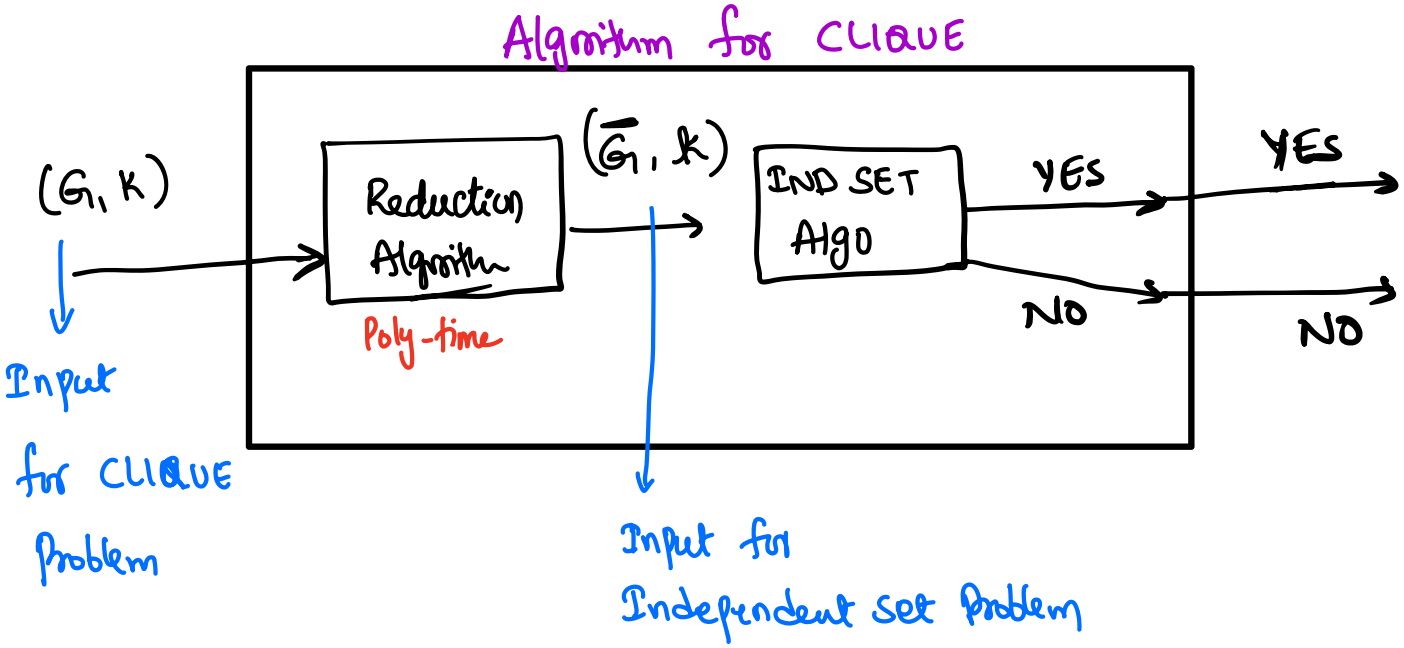
I is a maximum size clique of $\bar{G}$.

ie, $\alpha(G) = \omega(\bar{G})$

Suppose you have an algorithm A that solves Independent set Problem



Q: Can we use A to solve ~~VERTEX COVER~~ CLIQUE Problem.


Algorithm for CLIQUE
(G, K)
Reduction Algorithm
(Ḡ, k)
IND SET Algo
YES
YES
NO
NO
Poly-time
Input for CLIQUE Problem
Input for Independent set Problem

Summary So far

VERTEX COVER
IND SET
CLIQUE

} If you have a Polynomial time algorithm that solves one of them then we can also solve the other two.

we do not know of Polynomial-time algorithms for these three Problems (even today).

we cannot Prove that no Polynomial-time algorithm exists.

Same is true for many other Problems like

- Hamiltonian Path
- TSP
- 3-SAT
- longest path
- etc

} I will define them later

The good thing is :

Researchers working in this area have found a large class of problems with the following property.

"A polynomial time algorithm for any one of them would imply the existence of a polynomial time algorithm for all of them"

[These are NP-complete problems, formally we define it later]

Complexity classes P and NP

P = Set of all decision Problems that can be Solved in Polynomial time.

Examples: Sorting, Shortest Path, MST etc

NP = Set of all decision Problems for which a solution can be verified in Polynomial time.

Examples: Vertexcover, Indset, Clique, Sudoku,

(*) Clearly $P \subseteq NP$

Q: Is P equal to NP? [Unsolved Problem]

[One of Clay Mathematics Institute Millennium Prize Problems]

belief: $P \neq NP$

Means there are Problems in NP that are harder to Compute than to Verify: they could not be solved in Polynomial time, but the answer could be Verified in Polynomial time.

Polynomial-time reduction

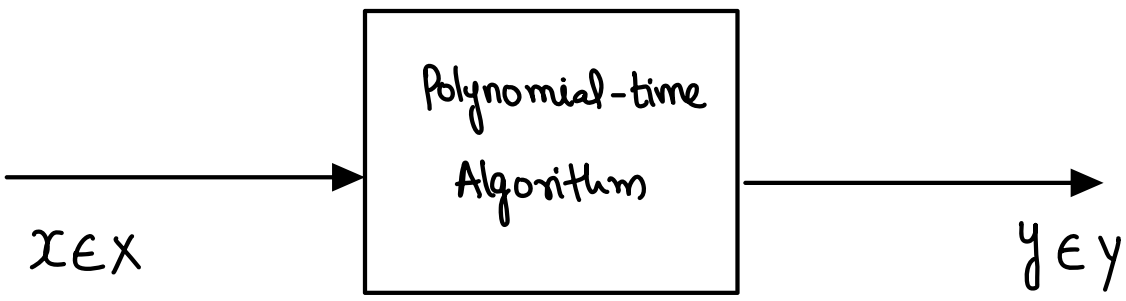
Polynomial-time reduction is a way of transforming one Problem into another Problem satisfying Some conditions.



What could be these conditions ?

Def: Polynomial-time reduction

We say that a problem X is polynomial-time reducible to a problem Y, denoted $X \leq_p Y$ if there exists a polynomial time algorithm such that given any instance x of X, the algorithm constructs an instance y of Y such that x is a YES instance of X iff y is a YES instance of Y.

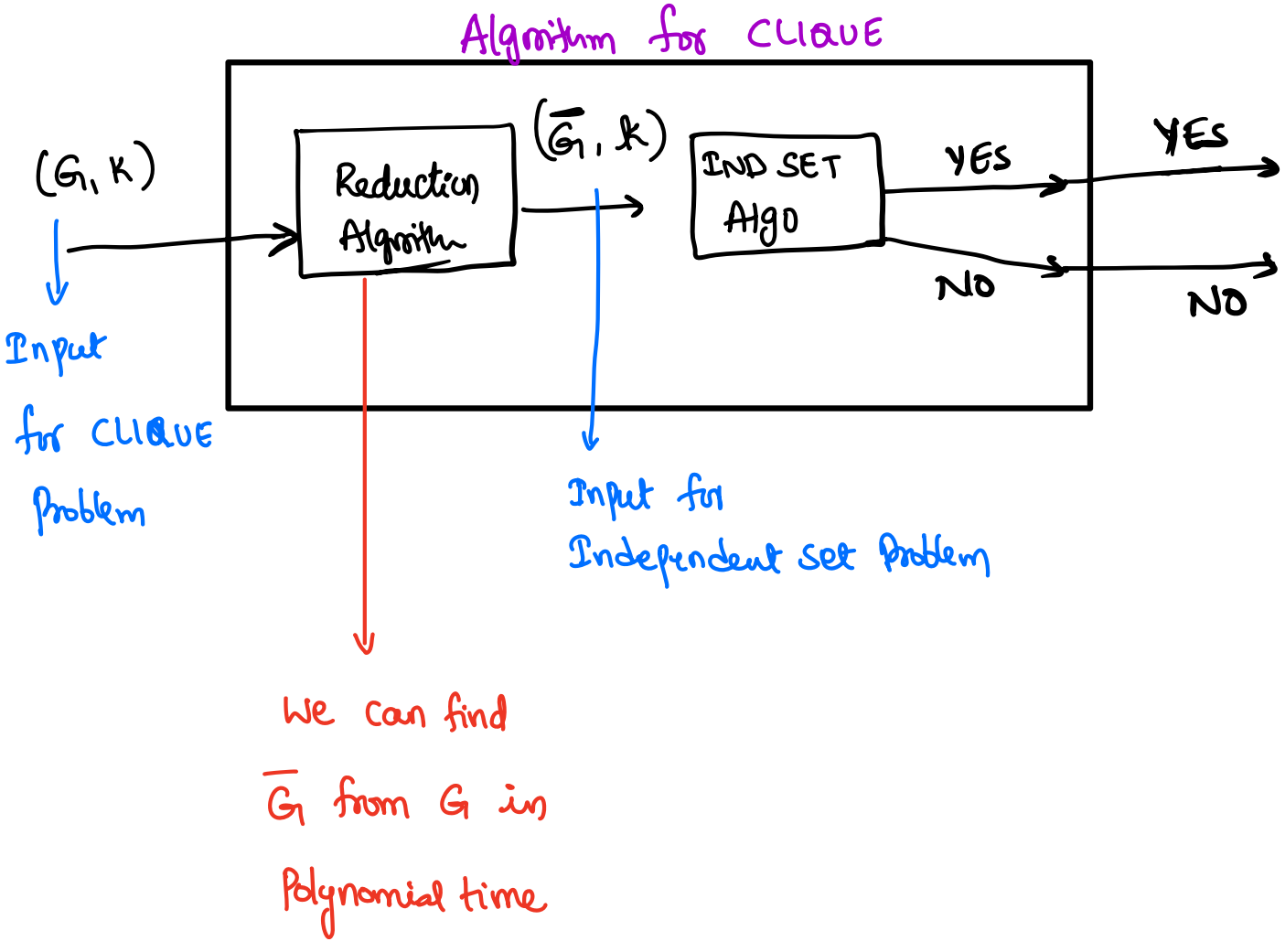


[Poly in fun of [I/p size]]

⊛ In general we show that

x is an YES instance of $X \iff y$ is an YES instance of Y

Polynomial time reduction



Set cover :

Given a set U of n elements, a collection $S_1, S_2, \ldots S_m$ of subsets of U and number k, does there exist a collection of at most k of these sets whose union is equal to all of U?

Example:

$$U = \{1,2,3,4,5,6\}$$

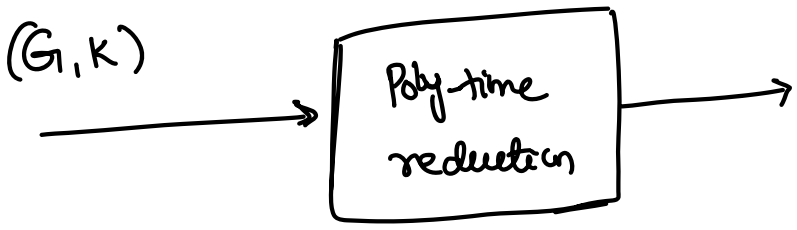
$$S_1 = \{1,3,5\}, \quad S_2 = \{2,3,4\}, \quad S_3 = \{2,4,6\}$$

$$k = 2$$

then $S_1 \cup S_3 = U$, hence it is a YES instance

Q: If we have a Polynomial-time algorithm for Set Cover, Can we use it to Solve Vertex Cover?

VERTEXCOVER $\leq_p$ SET-COVER



$U = E(G)$

$S_v = \{uv \in E(G) \mid u \in V(G)\}$

k

↓ set of edges incident on vertex v.

Claim: (G, K) is an YES instance of VC

$\Leftrightarrow$

$(U, S_{v_1} .. S_{v_n}, k)$ is an YES instance of SET-COVER

Boolean Formula

A Boolean formula is an expression built using

- Boolean variables (e.g., $x_1, x_2 \ldots$)
- Logical operators
  - AND $\wedge$
  - OR $\vee$
  - NOT $\neg$
- Parentheses

Examples

① $x_1 \wedge x_2$

② $(x_1 \vee x_2) \wedge \neg x_3$

③ $(x_1 \vee \neg x_2) \wedge (x_2 \wedge \neg x_3)$

A truth assignment for a boolean formula ϕ is a set of values for the variables of ϕ.

A satisfying assignment is a truth assignment that causes boolean formula to evaluate to 1.

Ex:

$$\phi = \big((x_1 \vee x_2) \wedge (\bar{x}_3 \vee \bar{x}_2) \big) \wedge \bar{x}_1$$

$x_1 = 1$
$x_2 = 1$
$x_3 = 1$

} Truth assignment

NOT a Satisfying assignment

$x_1 = 0$
$x_2 = 1$
$x_3 = 0$

} Truth assignment + a Satisfying assignment

A formula with a satisfying assignment is a satisfiable formula.

Eg ① $\phi = (x_1 \vee x_2) \wedge (\bar{x}_1 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_2)$

is satisfiable, $x_1 = 0$, $x_2 = 1$

② $\phi = (x_1 \vee x_2) \wedge (\bar{x}_1 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_2) \wedge (x_1 \vee \bar{x}_2)$

is NOT satisfiable.

Satisfiability Problem (SAT)

Ip: A Boolean formula ϕ

Q: Is ϕ satisfiable? (YES or NO)

A **literal** is a either a Propositional Variable or the negation of a Propositional Variable.

Negated Variables such as $\neg x$, $\neg y$ are called negative literals.

Some times we write $\bar{x}$, $\bar{y}$

A **clause** is a disjunction (OR) of one or more literals.

Eg: $(x \vee \bar{y} \vee z)$, $(\bar{x} \vee \bar{z})$

A Boolean formula is in conjuctive normal form or CNF, if it is expressed as an AND of clauses, each of which is the OR of one or more literals.

Eg:

$$\phi = (x_1 \vee x_2) \wedge (\bar{x_1} \vee x_3 \vee x_4 \vee x_5) \wedge (x_2 \vee x_4 \vee \bar{x_5})$$

$$= C_1 \wedge C_2 \wedge C_3$$

Where $C_i \rightarrow$ Clause

3-CNF

A Boolean formula is in 3-CNF if each clause has exactly three distinct literals.

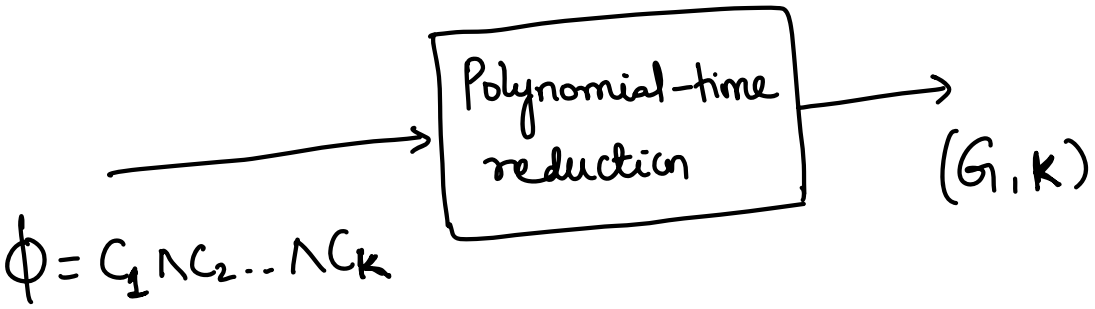
Eg: $\phi = (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_1 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee \bar{x}_3 \vee \bar{x}_4)$

3-SAT or 3-CNF-SAT Problem

Input: A Boolean formula ϕ in 3-CNF form

Q: Is ϕ satisfiable? (YES | NO)

3-CNF-SAT $\leq_p$ CLIQUE



$$\phi = C_1 \wedge C_2 \ldots \wedge C_k$$

Where each clause has exactly 3-literals

Construction of graph

(i) Vertices:

- For each literal in each clause, create one vertex

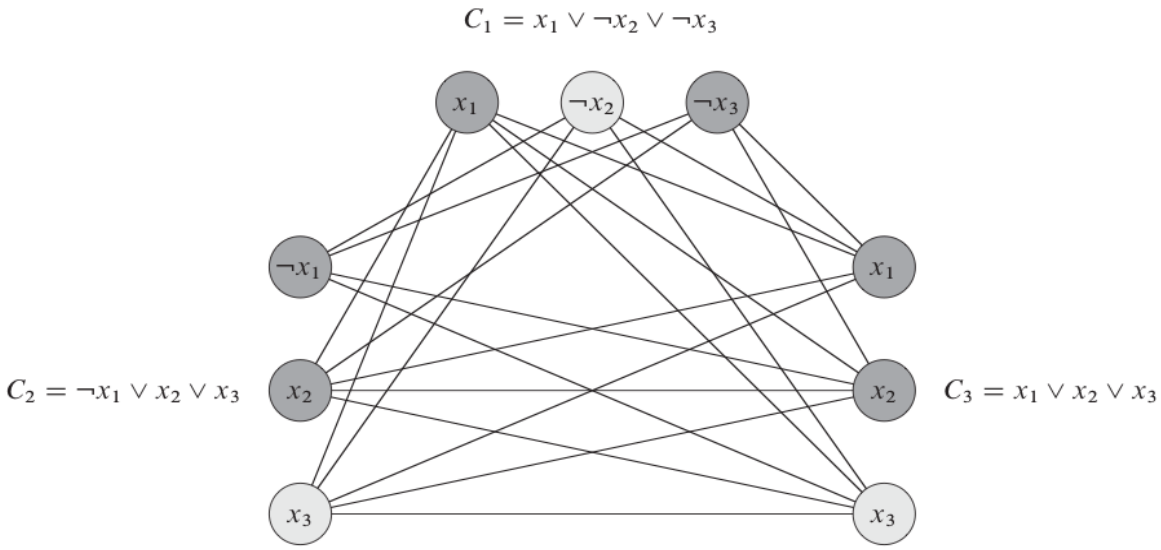
(ii) Edges:

- Connect vertices from different clauses only
- Add edge if literals are not contradictory

  ie, No edge b/w x & $\neg x$

Eg $\phi = (x_1 \vee \overline{x}_2 \vee \overline{x}_3) \wedge (\overline{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$

C_1 C_2 C_3



Claim: [Imp] [Equivalence]

ϕ is Satisfiable $\iff$ G has a Clique of Size k

NP-Complete :- A Problem X is NP-Complete if

① X is in NP

② $Y \leq_p X$ for every Problem $Y \in NP$

How to Show a Problem Π is NP-Complete?

(i) Show $\Pi \in NP$

- A given Solution can be verified in polynomial-time

(ii) From the definition, we need to show

$$\Pi' \leq_p \Pi \quad \text{for every } \Pi' \in NP$$

[Isn't it difficult]

TRUE | FALSE:

If Y is an NP-complete problem and X is a problem in NP with the property that $Y \leq_p X$ then X is NP-complete.

Q: How can we use the answer to the above true/false question to prove that a problem Π is NP-Complete?

Answer:

Take a known NP-complete problem Π' and reduce it to your problem

ie,

We need to prove $\Pi' \leq_p \Pi$

We need at least one NP-Complete Problem as a Starting Point, after that, we can use it to Prove other Problems are NP-Complete.

First NP-Complete Problem

SAT is the first problem that was proven to be NP-complete (See cook-Levin theorem)

[For proof refer to Michael Sipser text book]

NP-Complete :- A Problem X is NP-Complete if

① X is in NP

② $Y \leq_p X$ for every Problem $Y \in NP$

A Problem X satisfies Property ② but not necessarily Property ① we say that X is NP-hard

ie, A Problem X is NPC

if $X \in NP$ and X is NP-hard.

- There is no known algorithm that efficiently solves each SAT Problem and it is generally belived that no such algorithm exist, yet this belief has not been proven mathematically, and resolving the question of whether SAT has a Polynomial-time algorithm is equivalent to the P versus NP Problem, which is a famous open Problem in theory of Computing.

- NP-Completeness only refers to the run-time of the worst case instances. Many of the instances that occur in Practical applications can be solved much more quickly.

Lemma: Suppose $Y \leq_p X$. If Y cannot be solved in polynomial time, then X cannot be solved in polynomial time.

Lemma: Suppose X is an NP-Complete Problem.

X is Solvable in Polynomial time if and only if $P = NP$.

Lemma: If Y is an NP-Complete Problem and X is a Problem in NP with the Property that $Y \leq_p X$, then X is NP-Complete.

(2) We know 3-SAT $\leq_p$ CLIQUE & 3-SAT is NP-complete

So from the above lemma CLIQUE is NP-Complete.

Similarly we can show that VERTEX COVER and INDSET are NP-Complete.

General Strategy for Proving New Problem NP-Complete

Given a new Problem X, here is the basic Strategy for Proving it is NP-Complete

① Prove that $X \in NP$

② Choose a Problem Y that is known to be NP-complete.

③ Prove that $Y \leq_p X$.

We know that

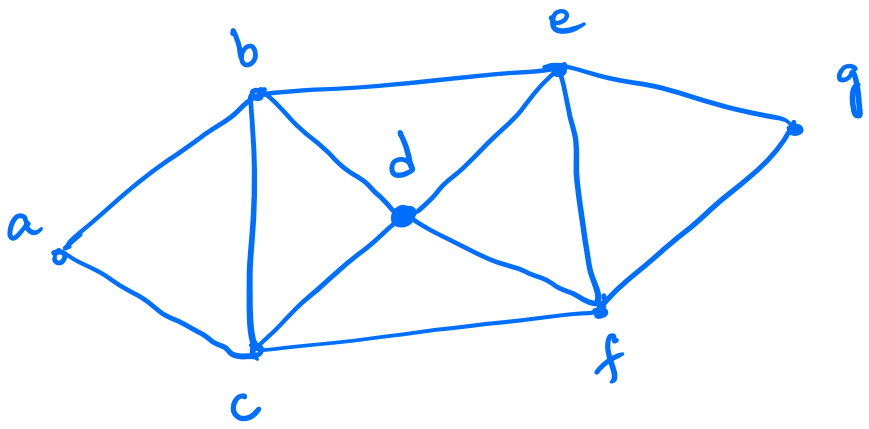
- 3-SAT is NP-Complete

- 3-SAT $\leq_p$ CLIQUE

Graph Coloring [Proper Coloring]

The graph coloring problem is to color the vertices of the graph using minimum # of colors such that adjacent vertices have the different colors.

Eg:



G

k-Coloring (or) Proper k-Coloring Problem

I/P: A graph G

Q: Does G have a k-coloring ?

↓

Coloring with at most k-colors

We show that 2-coloring can be solved in Polynomial time, however k-coloring for $k \geq 3$ is NP-Complete.

What are 2-Colorable graphs?

3-coloring Problem

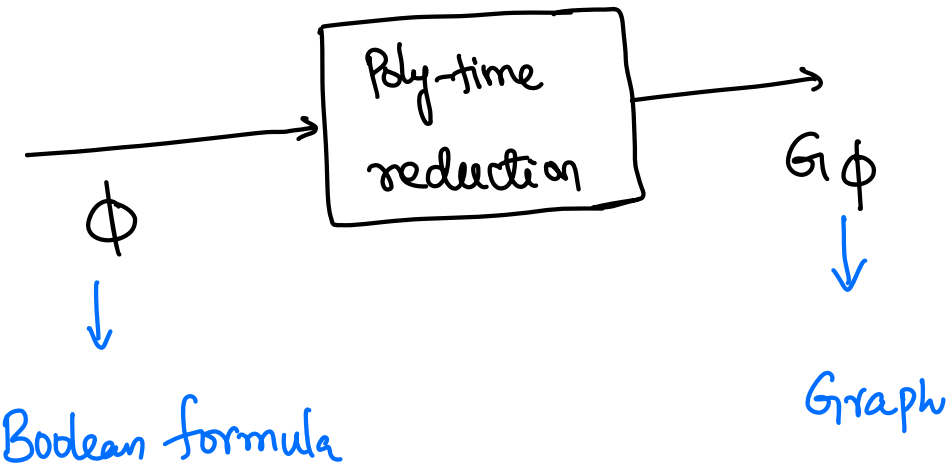
Input: A graph G.

Q: Does G have 3-coloring ?

* 3-coloring is in NP

Next we show that

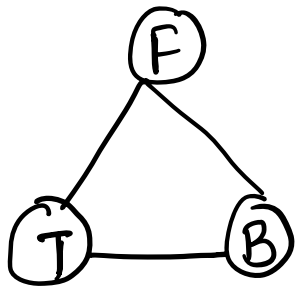
$3\text{-SAT} \leq_p 3\text{-coloring}$



On input ϕ, Construct G_ϕ as follows.

- Add 3 special nodes

  T = "true", F = "false", B = "base"

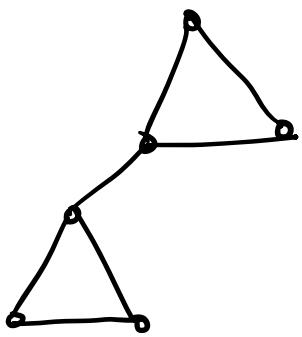


- For each variable

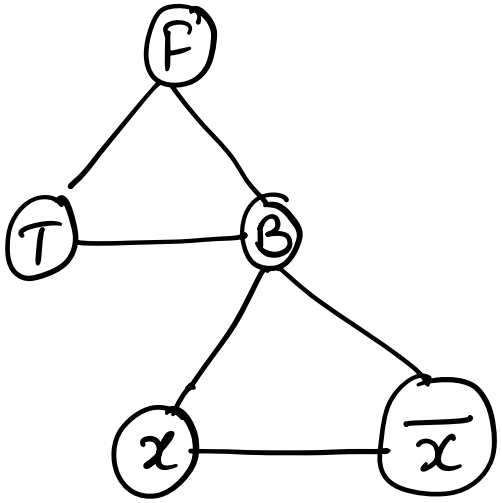
  add 2 literal nodes



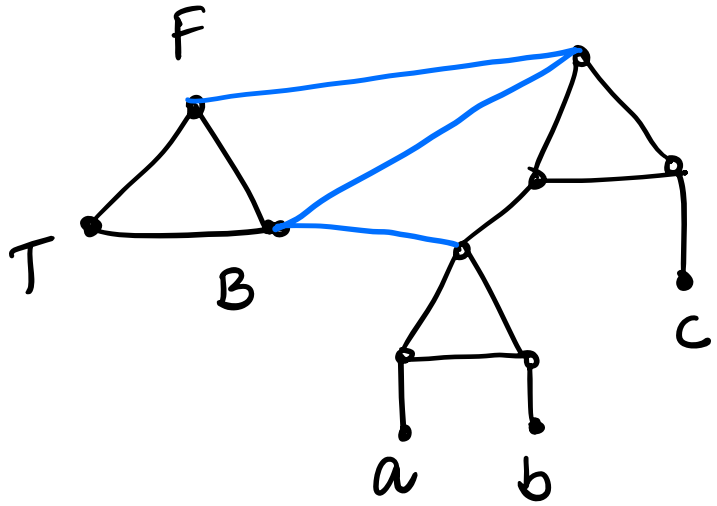
- For each clause add 6 clause nodes



- For each variable x connect

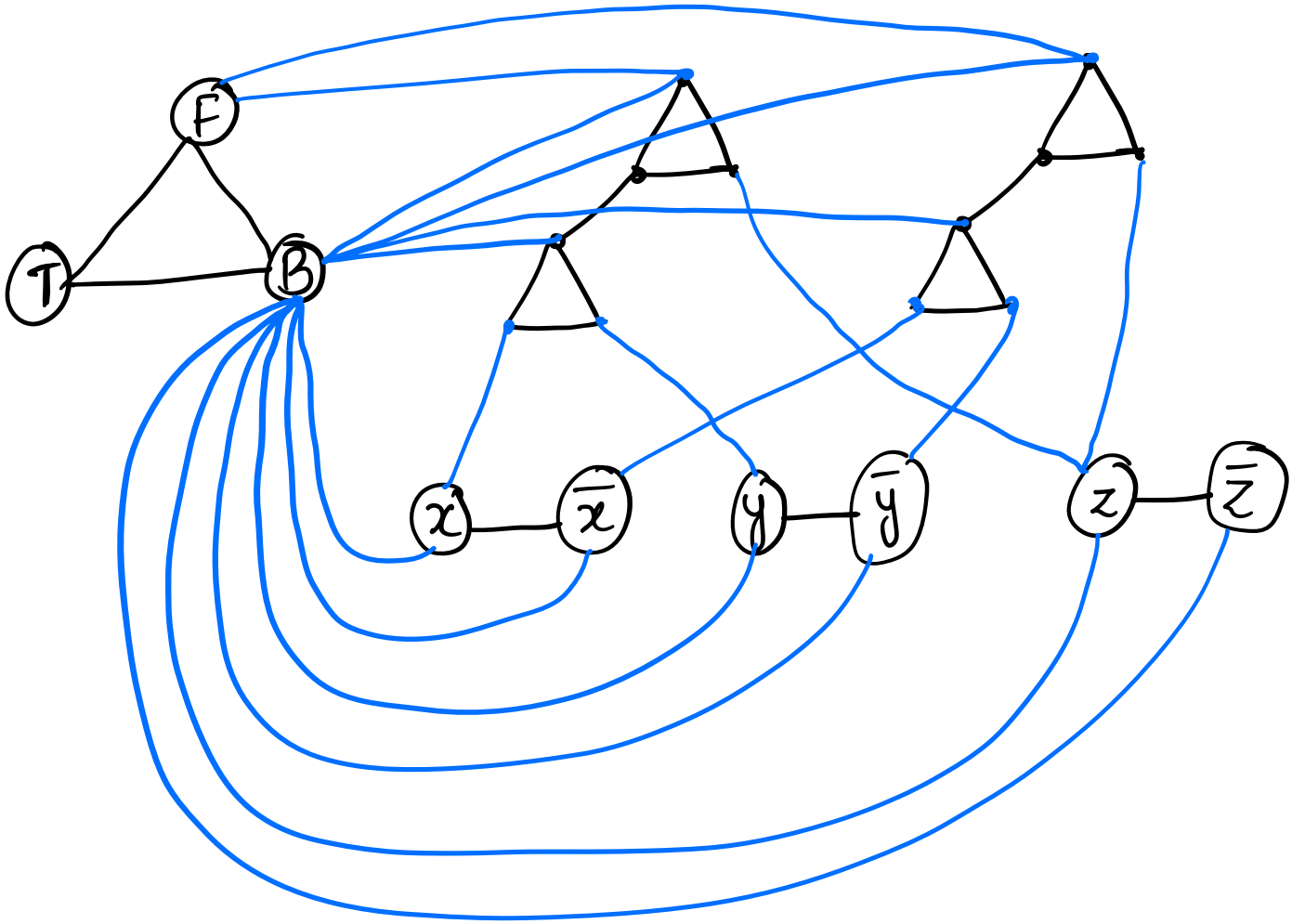


- For each clause $(a \vee b \vee c)$ connect



Clause - gadget

Example: $\phi = (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee z)$



Aim:

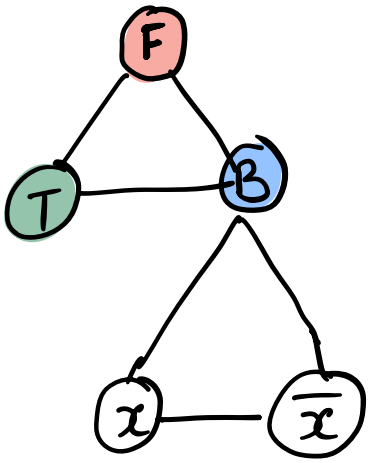
ϕ has a satisfying assignment

$\Leftrightarrow$

G_ϕ has a 3-coloring.

Some Remarks

In any 3-coloring of G, the nodes True, False and Base must get all three colors in some permutation. Thus we refer to the 3 colors as the True color, False color and the Base color based on which of these nodes gets which color.



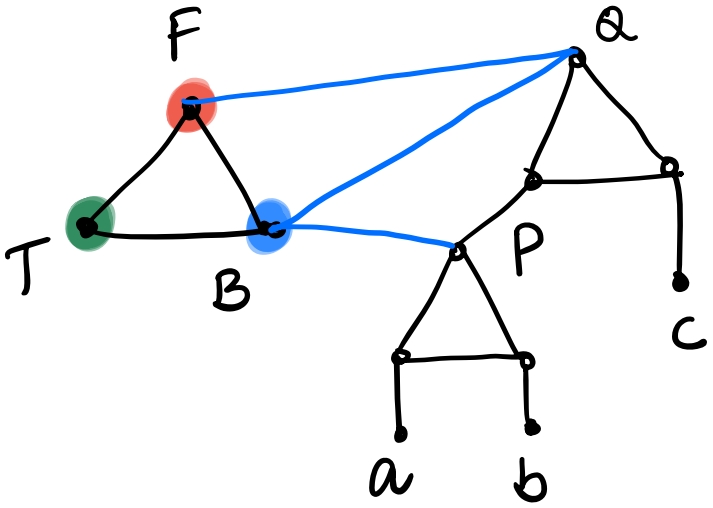
- In any 3-coloring, all variable nodes must be colored T or F, because they are adjacent to B.

Also x and $\bar{x}$ must have different colors as $x\bar{x} \in E(G\phi)$.

- So we can convert a 3-coloring of $G\phi$ into a true/false assignment to variables of ϕ.

Clause-gadgets

Fact: Graph below 3-colorable $\Leftrightarrow$ one of a, b, c colored T (ie, green)

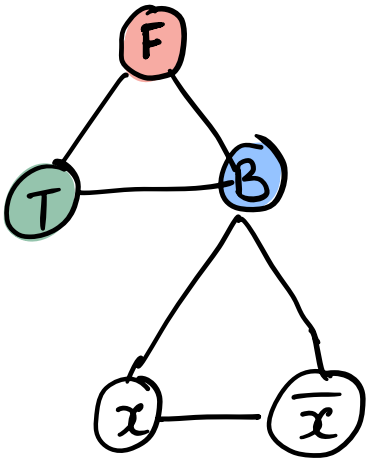


Claim: ϕ is Satisfiable $\Leftrightarrow$ G_ϕ is 3-colorable

Forward:

Suppose ϕ has a satisfying assignment

Color literal nodes T or F accordingly



For each i, assign x_i the true color if $x_i = 1$ and False color if $x_i = 0$ and then $\overline{x_i}$ is assigned to only available color

As argued above, it is now possible to extend this 3-coloring into clause gadgets resulting in a 3-coloring of G_ϕ.

Reverse:

Suppose G_ϕ has a 3-coloring,

In this 3-coloring, each node x_i is assigned either True color or False color.

then we set the variable x_i in ϕ correspondigly.

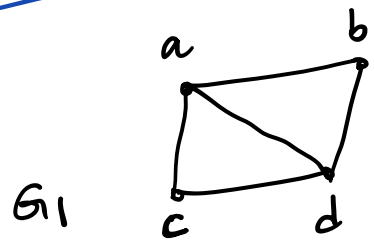
Claim: In Each clause of the 3-SAT instance, at least one of the terms in the clause has truth value 1.

If not, then all three of the corresponding nodes has the False color in the 3-coloring of G_ϕ and we have seen above there is no 3-coloring of the corresponding clause gadget consistent with this, which is a contradiction.

– 3-coloring $\leq_p$ k-coloring $(k>3)$

Def :- A Simple Path (cycle) in a graph containing all its vertices is called a **HAMILTON PATH** (respt. HAMILTON CYCLE).

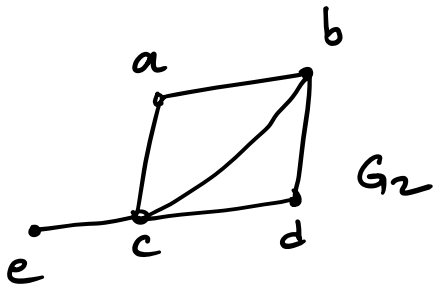
Examples :



G_1

$C = a\ b\ d\ c\ a$ is a Hamiton cycle

$\therefore$ G_1 is Hamiltonian



G_2

G_2 is NOT Hamiltonian

But G_2 has a Hamilton Path

$P = a\ b\ d\ c\ e$

Hamiltonian Cycle Problem

Given a graph G, check whether G is Hamiltonian or not.

Theorem:

The Hamiltonian Cycle Problem is NP-Complete.

Proof: Refer Theorem 34.13 on Page 1091 in Coreman et al. text book.

Theorem:

The Hamiltonian PATH Problem is NP-Complete.

Proof:

$$H.\text{cycle} \leq_p H.\text{Path}$$

Longest Path Problem

Longest Path Problem is the Problem of finding a Simple Path of maximum length in a given graph.

↓

doesn't have any repeated vertices.

"In contrast to the Shortest Path Problem, which can be solved in polynomial time in graphs without negative-weight cycles, the longest Path Problem (decision version) is NP-complete"

↓

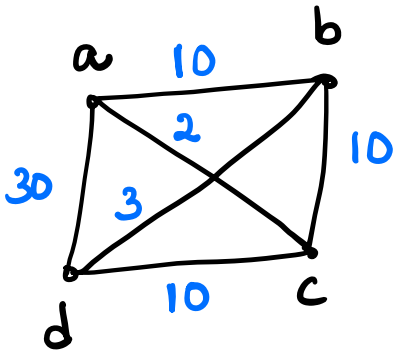
Input: A graph $G=(V,E)$, an integer k.

Question: Is there a Path of length at least k in G?

This Problem is NP-complete, as there is an obvious reduction from Hamiltonian Path Problem.

Travelling Salesman Problem

Example :



Hamiltonian cycle of minimum cost is

$a - b - d - c - a$ of cost $10 + 3 + 10 + 2 = 25$

Decision Version of TSP

I/p: Complete graph G, $c : V \times V \to \mathbb{Z}$, $k \in \mathbb{Z}$

Q: Does G has a Hamiltonian cycle with cost at most k.

TSP is NP-Complete

Remark: TSP is NP-Complete

Proof Idea: Hamiltonian cycle $\leq_p$ TSP

Let $G_1 = (V, E)$ be an instance of Hamiltonian cycle problem. We construct an instance of TSP as follows.

We form a complete graph $G' = (V, E')$, where $E' = \{ uv \mid u \in V, v \in V \}$. Cost function c is defined as

$$c_{uv} = \begin{cases} 0 & \text{if } uv \in E \\ 1 & \text{otherwise} \end{cases}$$

The instance of TSP is then $(G', c, 0)$ which can be constructed in Polynomial time.

Claim: The graph G has a hamiltonian cycle if and only if graph G' has tour of cost at most Zero.

Proof: Exercise

We will revisit TSP in approximation algorithms Part.

Continuing along these lines we can show that many problems are NP-Complete, for example

Hamiltonian cycle/path

Travelling Salesman

Subset sum

Setcover

Hitting set

- - - - -

Summary

No Polynomial time algorithm is known for any NP-Complete Problem. If a Polynomial time algorithm were found for any one of the NP-Complete Problem, we would able to construct Polynomial time algorithms for all of them.

Researchers belive that none of the NP-Complete Problems can be solved in Polynomial time. However this has not Proven mathematically.